Claim Denial Triage Agent for Outpatient Revenue Cycle

Hriday Jyoti Pradhan (hpradha3@asu.edu), Neha Kashyap (nkashya6@asu.edu), Silas Sangmin (ssangmin@asu.edu), and Sachin Kothandaraman (skothan4@asu.edu).

Stakeholder name: Mokshitha Mohan Meeting date: 11/7/2026

Section 1 Stakeholder Follow-Up and Final Deliverables

Format and participants: The review was asynchronous. On November 7, 2026 (10:10 PM), Silas posted a review link in the team's group chat with the stakeholder, Mokshitha. The link covered progress since Milestone 1, a real system output, questions, six test cases for her to check, and a sign-off section. Mokshitha acknowledged it the next morning (9:39 AM) and returned written answers on November 8, 2026.
Participants: Mokshitha and team members Silas, Neha, Hriday, and Sachin.

Presented and received. We presented the Milestone 1 to 2 progress summary, two example outputs (rule-based and tool-trace), and six test cases for her to check against her own judgment.

Costliest denial. The stakeholder named medical necessity the costliest denial type, because staff must pull the chart, find the policy, and match them line by line, usually for expensive imaging, whereas coding errors are more frequent but quick fixes. → **Keep:** this confirms the project's focus on evidence-dependent medical-necessity review. → **Next:** continue prioritizing policy and encounter-note retrieval with criterion-level assessment.

Output readability. She said "Criterion A: SUPPORTED" plus a note ID tells a specialist nothing, and asked for the criterion text, the actual note line, the matched policy passage, and the deadline and approval status at the top. → **Change:** the passages are already in the per-run evidence store, so this is a presentation fix. → **Next:** revise the OutputPackage report to show those fields under a plain-language header.

Real model. She named a real model as the top priority, noting that pilot numbers on scripted responses "won't really mean much." → **Keep.** → **Next:** complete the hosted transport (#34) and label pilot results as development checks until then.

Case 3, no prior auth with retro window open. She disagreed with manual review and wants retro auth requested immediately and flagged urgent with the window end date, since the window is in the payer profile and delay risks losing the claim. → **Change:** no new policy corpus is needed. → **Next:** add urgent, deadline-flagged routing in the routing and output layer, keeping the baseline frozen for comparison.

Case 4, coverage ended. She agreed with billing the patient but asked that coverage be re-verified first, since eligibility checks are deferred. → **Change:** this resolves #30 in favor of a checklist step rather than a retrieval tool. → **Next:** add a manual re-verification step to the action plan.

Case 5, write-off. She agreed only for CO codes, since PR denials can usually be billed to the patient. → **Change.** → **Next:** check the group code before recommending a write-off.

Case 6, deadline passed. She agreed with manual review but asked to see days expired and whether the payer accepts exceptions. → **Change.** → **Next:** compute days expired deterministically and show exception availability only if the payer profile has the field; otherwise, "not available."

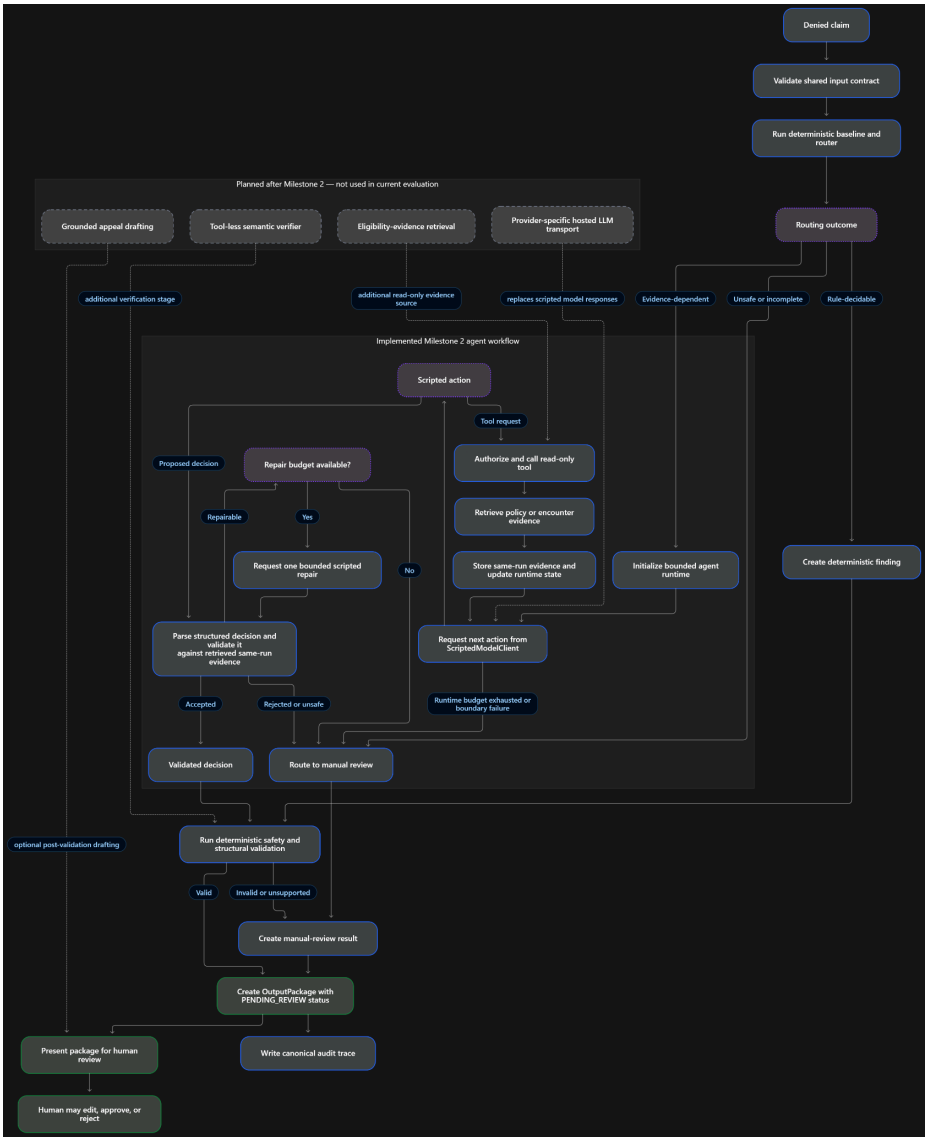
Case 2, bundling. She agreed with manual review for now, noting NCCI edits are public and could be added later. → **Defer.** → **Next:** keep bundling in manual review and list NCCI as future work.

Letters after evidence check. She agreed that appeal letters should follow the evidence check. → **Keep.** → **Next:** grounded drafting stays after tool-less evidence verification.

Agreed scope: Deliverables are a review-package generator that never submits claims or appeals and starts every package as PENDING_REVIEW, the unchanged baseline, the evaluation harness and results, and the final report. Expected functionality covers root-cause and disposition triage, policy and note retrieval with resolvable citations, criterion-level assessment with deterministic validation, deadline calculation, a specialist-readable report, and conservative manual-review fallback. Success criteria are the planned metrics (joint root-cause/disposition accuracy, selective accuracy, resolution coverage, citation precision, unsupported assertions, consistency, latency, cost, critical safety failures) plus stakeholder-driven checks: a specialist can understand the report without opening source files, and no deadline-sensitive case goes unflagged. Out of scope are NCCI edits, live payer or EHR integration, claim submission, and eligibility retrieval beyond the re-verification step.

Section 2 Implemented Agentic System

System and workflow. The Claim Denial Triage Agent is a hybrid system that turns a denied outpatient claim into a human-review package for an accounts-receivable specialist. It never submits claims or appeals, and every output starts as PENDING_REVIEW. A synthetic claim JSON is validated and run through the frozen deterministic baseline (code mapping, payer profile, deadline, disposition). Rule-decidable cases bypass the model. For evidence-dependent cases (e.g., CO-50 medical necessity), a bounded plan–act–observe loop retrieves the matching policy and the claim's encounter note, then proposes a structured AgentDecision with criterion-level assessments and citations. A deterministic safety validator checks the decision, producing an OutputPackage and a canonical JSON Lines trace. Eligibility retrieval, hosted transport, tool-less evidence verification, and grounded drafting are planned. Since Milestone 1, the scripted offline runtime became a production-shaped integration with real local tools, a claim-bound dispatcher guard, and durable traces; the corpus grew from v0 to v1 with Dataset B encounter notes; and a provider-neutral HostedModelClient was added, with live transport still pending (#34).



Roles and decisions. This is a single agent with narrow authority. The model is used only to interpret evidence (does a note satisfy a criterion); input validation, deadline arithmetic, permitted dispositions, citation checks, formula evaluation, and approval state stay deterministic, so model errors cannot silently change a high-stakes outcome. Multi-agent coordination was not adopted: the task is a single sequential evidence chain, and additional coordination would undermine auditability without clear benefit.

Planning and execution. The runtime has explicit phases and controlled transitions, with configurable limits on model-interface calls, tool calls, repairs, total steps, and elapsed time. For the Milestone 2 evaluation, each case was limited to four model-interface calls, three tool calls, one repair attempt, seven total model/tool steps, and 30 seconds. Each step reads and updates the evidence context, which determines the next permitted action or whether a decision may be proposed.

Tools and function calling. Tools are typed, read-only, and return one shared ToolResult contract. ToolDispatcher validates requests, enforces the route-derived allowlist, and applies a claim-bound guard to ensure that the policy payer, procedure, service date, and encounter claim ID match the active claim. Unknown codes or unlisted payers return NO_MATCH rather than a default profile. Model outputs are strictly parsed, and anything outside the schema is rejected.

State and knowledge. Memory is per-run only: a same-run evidence store plus append-only trace entries, with no long-term memory, preventing cross-claim leakage. Knowledge is a hash-pinned, versioned corpus (knowledge-v1) with stable passage IDs and dated policy versions (service date selects PT policy v1 or v2), so runs are reproducible and corpus changes detectable.

Verification and recovery. Validation runs during execution. Citations must resolve to retrieved passages, and policy formulas (e.g., A AND B AND (C OR D)) are parsed by a purpose-built Boolean parser (never executable evaluation; malformed formulas fail closed). Malformed or rejected output triggers a budget-limited repair prompt carrying structured problem codes, field paths, messages, and repairability flags (one attempt in the demo). Exhausted budgets, missing or conflicting evidence, or a hosted timeout result in an audited, conservative MANUAL_REVIEW package. Retrieved text is treated as untrusted data and tested using an encounter note containing instruction-like text.

Models and services. Python 3.9+, Pydantic contracts, and BM25 retrieval over local files. The model is currently a deterministic scripted fake, and HostedModelClient is tested only with a fake transport, so no external API, credentials, or network are used. The hosted provider is not yet selected or committed.

Section 3 Preliminary Evaluation Results and Analysis

Evaluation setup. This is a synthetic development pilot, not the final held-out evaluation. All values come from committed artifacts in evaluation/results/m2-pilot/ (commit 62be84d) and can be reproduced with python -m evaluation.compare_pilot. Each case is one denied service line, labeled under a rubric that was frozen before any system output was scored. Two team members labeled every case independently, and a third adjudicated their disagreements.

Datasets. Dataset A (rule and safety triage, 23 cases) checks that the system does not regress on denials the rules already handle and that it escalates unsafe ones. Its 21 synthetic claims cover all 10 root causes: unmapped code, unknown payer, expired deadlines, and mixed CO/PR lines. Its labelers agreed on 19 of 23 dispositions ($\kappa = 0.79$). Dataset B (evidence-dependent policy review, 12 cases) targets medical necessity, which the stakeholder named the costliest denial. Its CO-50 claims on corpus knowledge-v1 cover MRI and physical therapy across two dated policy versions, conflicting notes, an injected instruction, and missing policies. Its labelers agreed on all 12 cases ($\kappa = 1.00$), which is expected because each case was designed around one deciding fact.

Systems and conditions. All three systems ran on the same 35 cases with an as-of date of 2026-09-06. S0, the current agent, runs the router, the bounded runtime, the validator, the output package, and canonical traces. It uses scripted-model-v1 at temperature 0, whose responses were scripted per case after labeling. It has budgets for 4 model calls, 3 tool calls, 1 repair, and 7 steps, and access only to the policy and encounter-note tools. S1 is the frozen, rule-based baseline, which the team would rely on without the agent. S2 uses S0's router but sends every agent-routed case to manual review, so the gap between S0 and S2 isolates what the agent adds.

Metrics. We report joint accuracy (root cause and disposition both correct), resolution coverage, selective accuracy, exact deadline accuracy, and critical safety errors from rubric §7, whose target is zero. Every rate carries a 95% Wilson interval in the artifacts, and S0's tool and citation behavior is read from its traces. Some measurements are reported as not available rather than zero. S1 and S2 write no traces, so trace-based safety checks don't apply to them. S0's resistance to the injected instruction is scripted, so it is not evidence either way. Latency, tokens, cost, and run-to-run consistency are not meaningful for a scripted model and are deferred to the hosted model (#34).

Metric	A: S0	A: S1	A: S2	B: S0	B: S1	B: S2
Joint accuracy	20/23 (87%)	19/23 (83%)	20/23 (87%)	12/12 (100%)	5/12 (42%)	6/12 (50%)
Resolution coverage	13/14	14/14	13/14	6/6	4/6	0/6
Selective accuracy	13/15	14/18	13/15	6/6	4/11	n/a
Deadline accuracy	6/6	6/6	6/6	4/4	4/4	n/a
Unsupported appeals	0	2	0	0	7	0
Missed escalations	2	3	2	0	4	0

Table 3.1: Results by dataset (correct/total; safety errors are counts).

On Dataset B, S0's 11 agent runs produced 6 decisions accepted by the validator and 5 manual reviews. Every tool call was valid (24 of 24), and every policy request matched the claim's payer, procedure, and service date (11 of 11). All 43 citations came from passages retrieved in the same run, and no run exceeded a budget, needed a repair, or failed.

Analysis. The clearest difference is on Dataset B. S1 cannot read evidence, so it appeals 11 of 12 cases and makes 11 critical safety errors. S2 is safe but resolves nothing. S0 resolves all 6 recoverable cases and correctly escalates the other 6. Because S0's responses are scripted, this shows that the pipeline carries a correct decision through retrieval, validation, and tracing without loss. It does not yet show that a model would make that decision, a point the stakeholder also raised.

On Dataset A, S0 and S2 tie at 20 of 23 and make fewer safety errors than S1 (2 against 5). They do this by escalating two disputes that have no applicable policy in the corpus (610102 and 610110), where S1 files unsupported appeals. They also show one regression: an unknown-payer contractual case (610116) is escalated unnecessarily. In addition, all three systems miss two escalations, which Section 5 examines.

Against the Milestone 1 targets, deadline accuracy was 6/6 on Dataset A and 4/4 on Dataset B among cases where the predicted disposition matched the adjudicated disposition, and rule-case accuracy did not fall below the baseline. The +20-point evidence target is met only with scripted responses. Zero safety errors and at least 90% selective accuracy (87% on Dataset A) are not yet met. With only 23 and 12 cases, most differences have overlapping confidence intervals; only S1’s errors on Dataset B clearly stand out.

Section 4 Demo or Case Study

A medical-necessity denial for a brain MRI

Task and context. Claim CLM-2026-005097 is a synthetic CMS-1500 claim billed to BCBS Arizona. It is for an MRI of the brain without and with contrast (CPT 70553, $1,450), with a service date of June 2, 2026. The payer denied it with CO-50 and remark M127, meaning the service was not considered medically necessary. The applicable policy, POL-BCBSAZ-MRI-BRAIN@v1, requires the formula A (headache with a neurologic red flag) AND B(a documented neurologic exam) AND (C(a contrast indication) OR D(prior non-diagnostic imaging)) for 70553.

Expected outcome: an appeal with documentation, supported by cited evidence, due January 7, 2027, and waiting for human approval.

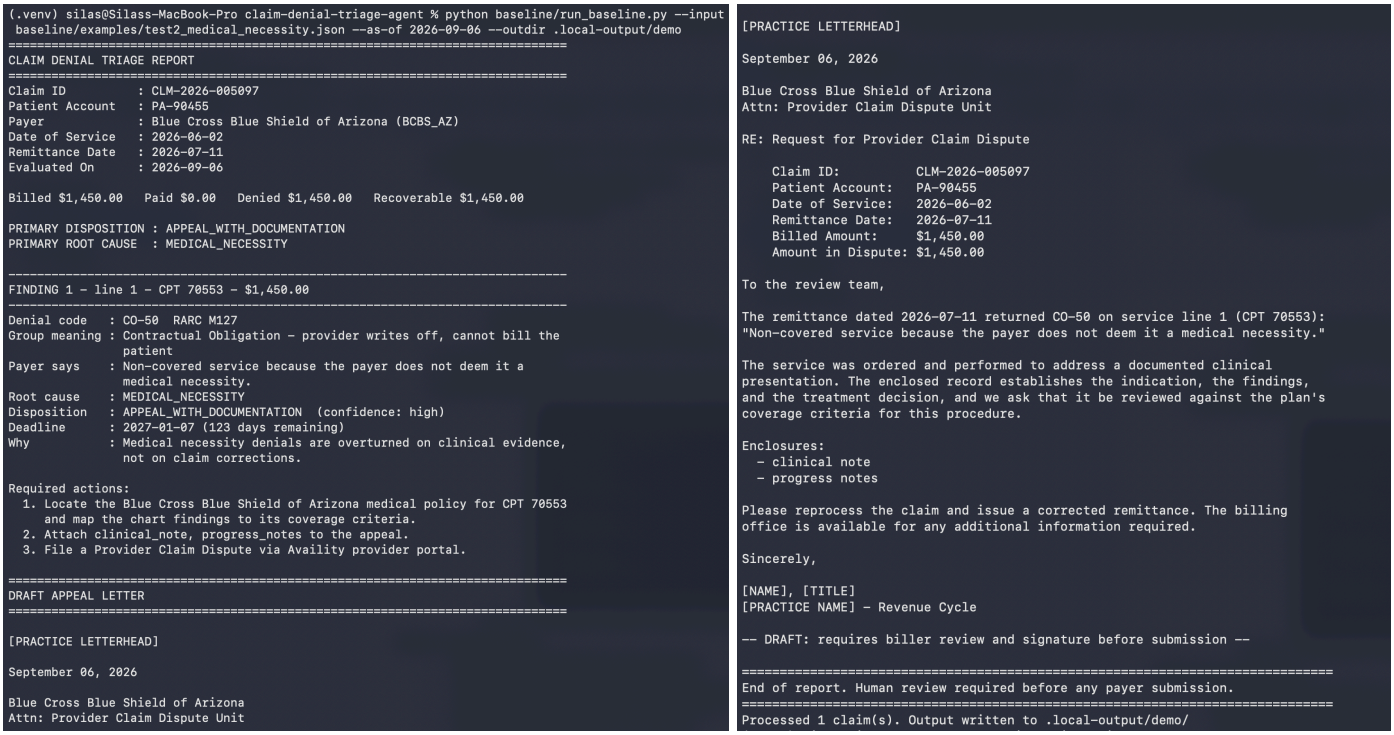


Figure 4.1: Baseline view of the case, showing the input and the expected outcome.

Run setup. The run used knowledge snapshot knowledge-v1. The model was scripted-model-v1, which replays pre-written model responses. The routing, tools, validator and output package are the real implementation. The budgets were 3 model calls, 2 tool calls, 5 total steps and 1 repair. The only tools allowed were retrieve_medical_policy and get_encounter_note.



Figure 4.2: Agent run, showing the budgets used, the policy formula and the ordered trace of 14 events.

Walkthrough:

1. **Routing.** The baseline maps CO-50 to MEDICAL_NECESSITY and calculates the appeal deadline as January 7, 2027: the July 11, 2026 remittance plus the payer's 180-day window. An appeal of this kind needs evidence the rules cannot read, so the router sends the case to the agent and allows only the two tools above.

2. **Policy retrieval.** The model calls retrieve_medical_policy with payer BCBS_AZ, procedure 70553 and service date 2026-06-02. A claim guard checks that those arguments match the active claim before the tool runs. The tool returns 7 passages, ranked by BM25 with P3 to P5 first, along with the formula.

3. **State update and next step.** The retrieved passages are added to the run's evidence and shown to the model on its next turn. The model then asks for the claim's neurology progress note, and get_encounter_note returns passages L1 to L5.



Figure 4.3: Tool calls, showing the arguments requested by the deterministic scripted model and the passages each tool returned.

4. **Decision.** The model marks A, B and C as SUPPORTED, each with an encounter-note citation, and proposes an appeal.

5. **Verification.** The deterministic validator checks whether the claim and line identity match, the deadline is still open, every citation was retrieved in this run, each supported criterion cites the encounter note, and the formula evaluates to true when parsed by the Boolean parser. If so, the decision is ACCEPTED.

6. **Output.** The package recommends APPEAL_WITH_DOCUMENTATION with no manual-review reasons, and it stays PENDING_REVIEW. The run used 3 model calls, 2 tool calls and 0 repairs.



Figure 4.4: Final decision, deadline, validation result and criterion citations.

Agentic mechanisms shown: Planning (each deterministic scripted model response requests the next permitted action) , State(retrieved evidence carries over between turns), Tool calling (every call goes through a claim-bound dispatcher), Verification(a deterministic validator checks the decision), Budgets and human approval(both are enforced), Recovery(not exercised in this run (see Section 5))

Does it meet the need? Yes, for the core task. The specialist gets a cited appeal recommendation they can verify, with the correct deadline. It doesn't yet include an appeal letter, and the output is JSON rather than a readable report. The run also exposes a gap. The agent cited the consultant's exam for B, but the policy requires the ordering clinician's exam, which is in the clinical note the agent never fetched. The validator can't catch this because it checks that citations exist, not what they mean. That is the reason for the planned evidence verifier in #34 (see Section 5).

Section 5 Failure Analysis and Current Limitations

Failure case: CLM-2026-610101-L1 (missed escalation). This claim has an office visit (99213, no modifier) that was bundled with a same-day knee injection and denied as CO-97. All three systems return CORRECT_AND_RESUBMIT with a deadline of 2027-07-02. S0's output package (set_a-scripted-agent/output_packages/CLM-2026-610101-L1.json) records route_outcome: DETERMINISTIC_ONLY and no manual-review reasons, so the agent never ran. The expected answer is MANUAL_REVIEW for missing evidence. Resubmitting with modifier 25 would assert that the visit was a separate service, and nothing in the case shows that. Both labelers and the adjudicator agreed. Case 610107 fails the same way: the system recommends routing to "the correct payer" even though no primary payer or EOB is on file.

Observed facts and suspected cause. The preserved artifacts show that each baseline finding lists the evidence its action depends on (ncci_edit_lookup for 610101; primary_eob and cob_confirmation for 610107), and that none of it is on either claim. Our diagnosis, which has not yet been tested, is that the router sends only appeals to the agent. It accepts every other confident deterministic disposition without checking whether that evidence is present, so the failure happens at routing, before the agent is ever involved.

Proposed fix and verification. The router should escalate a recoverable, deterministic finding when the evidence justifying the action is missing. It must ignore items the action itself supplies, such as the field being corrected, to avoid over-escalating correct answers. Hriday will make the change, with Neha reviewing the safety behavior. We will verify it by rerunning the comparison. 610101 and 610107 should be moved to manual review, and no other cases should change. Completion evidence is new router tests and zero missed escalations on Dataset A, followed by confirmation on the held-out set.

Partial success in the Section 4 demo. The validator accepted the demo decision, but its citation for criterion B points to the consultant's exam. The policy requires an exam by the ordering clinician, and that exam is in a note the agent never retrieved. The validator checks that each citation exists and was retrieved in the run, not what it means. So the 43-of-43 citation result measures provenance, not relevance. The tool-less evidence verifier planned in #34 is meant to close this gap.

Current limitations. The largest limitation is the scripted model: the pilot verifies the pipeline, but not model reasoning, consistency, latency, or cost. The data is also small and synthetic, with 35 development cases and no held-out set yet; the final evaluation targets 80 held-out cases. The corpus covers only two payer policies and has no bundling, authorization, or eligibility policies, so those disputes escalate by design. No pilot run experienced repair, budget exhaustion, or tool failure, which are currently covered only by fault-injection tests. Several optional tools remain unintegrated: code and payer lookups (#28), an agent deadline tool (#29), and eligibility retrieval (#30). One labeling choice also differs from stakeholder practice: she wants an urgent retro-auth request where the rubric currently says manual review (610110). Finally, the system has no live payer or EHR integration, and all outputs remain PENDING_REVIEW for human approval.

Section 6 Remaining Work Before the Final Submission

Milestone 2 establishes a reproducible local agent workflow, two-dataset pilot evaluation, and comparison against two baselines. The remaining work focuses on replacing temporary model boundaries, strengthening evidence-grounded output generation, expanding evaluation, and preparing the final demonstration. Required final-project work is prioritized ahead of optional system extensions.

Priority	Current gap	Improvement or experiment	Owner	Completion evidence
P0	The Milestone 2 agent uses deterministic scripted responses rather than a live provider-specific model. Issue #34 tracks the remaining model work.	Select the provider and model version; implement the provider-specific transport behind the existing model interface; freeze the prompt, parameters, budgets, timeouts, and credential convention; and record model identity, latency, token usage, and configuration hashes.	Hriday	An evidence-dependent case completes through the hosted-model path; provider failures safely produce MANUAL_REVIEW; the frozen configuration and canonical trace are preserved; and automated tests continue to run without hosted credentials.
P0	The validator checks deterministic structure and citation provenance but does not independently verify whether the cited evidence semantically supports the proposed decision.	Add a tool-less verifier that receives only the proposed decision and evidence retrieved during the same run. Require structured verification results and route uncertainty, disagreement, malformed output, or provider failure to human review.	Hriday, with Neha reviewing the safety boundary	Tests demonstrate that the verifier cannot access tools or introduce new evidence, unsupported decisions reach MANUAL_REVIEW, and the verification event appears in the canonical trace.
P0	The system produces a review package but does not yet generate a grounded appeal draft through the integrated agent workflow.	Generate a draft only after deterministic validation and semantic verification succeed. Use only the validated decision and resolvable same-run evidence, preserve citation identifiers, and keep every draft PENDING_REVIEW.	Hriday, with Neha reviewing approval safety and Silas reviewing citation provenance	Tests show that drafting is blocked for rejected and manual-review outcomes, cannot introduce unsupported evidence, retains resolvable citations, and never automatically submits an appeal.
P0	Milestone 2 reports development-pilot results rather than a final held-out evaluation.	Freeze the final code, prompt, model configuration, knowledge snapshot, datasets, labels, and scoring procedure. Run S0, S1, and S2 on the held-out set without tuning, and compare their results.	Sachin, with inputs from all component owners	Preserved raw outputs and traces; final system-by-dataset tables; safety-error counts; category-level analysis; consistency, latency, token, and cost measurements where available; and documented limitations.
P1	The stakeholder requested an output that can be understood without opening the underlying source files, and the final demonstration package has not yet been assembled.	Revise the specialist-facing report to show a plain-language finding, matched policy and passage, deadline or days expired, approval status, and evidence summary. Select representative success and failure cases, capture readable traces and output packages, update the architecture figure, and rehearse the final demonstration.	Team: final editor and demonstration presenter to be assigned	Stakeholder-readable output; reviewed report and slides; final demonstration script and screenshots; reproducibility commands; contribution record; submission branch, commit SHA, and release tag.

P1	Denial-code lookup, payer-profile lookup, an agent-callable deadline wrapper, and eligibility retrieval are implemented, incomplete, or deferred at different levels.	For each extension, either approve its route authorization and integrate it with least-privilege, claim-bound behavior, or record an explicit final-scope deferral. Eligibility retrieval remains optional unless retained for the final demonstration.	Neha for routing and safety; Silas for retrieval boundaries; Hriday for runtime integration; Sachin for evaluation	Focused tests cover authorization, rejection of unauthorized or mismatched requests, provenance, traceability, and safe failure; or the final documentation records an explicit deferral.

Provider-specific execution, semantic verification, grounded drafting, the held-out evaluation, and the stakeholder-readable demonstration package are required before the final submission. Corpus expansion, optional dispatcher and retrieval tools, adversarial or fault-injection experiments, and live payer, clearinghouse, EHR, or claim-submission integration are optional or future-phase extensions. They should be implemented only when they support the agreed final use cases; otherwise, the final report will document their deferral, rationale, and resulting limitations.

Section 7 GitHub Repository and Team Contributions

GitHub Link: https://github.com/hridaypradhan/claim-denial-triage-agent.git
Submission Branch and Commit: main, 65e858dd091d2e4124346c73c622f1308414171a

Setup and execution

Clone the repository, enter its root directory, and install the required Python dependencies:

git clone https://github.com/hridaypradhan/claim-denial-triage-agent.git

cd claim-denial-triage-agent

python -m pip install -r requirements.txt

Set the source-module path for the current terminal session.

Windows PowerShell: *$env:PYTHONPATH = '.\src'*
macOS or Linux using Bash/Zsh: *export PYTHONPATH=./src*

Then run the same commands on Windows, macOS, or Linux:

python -m evaluation.compare_pilot

python -m unittest discover -s tests/evaluation -p "test.py" -q*

If Python is exposed as `python3` on macOS/Linux or `py -3` on Windows, substitute that launcher for `python`. The comparison command runs S0, S1, and S2 on both adjudicated development datasets and writes the combined report to:

.local-output/eval/m2-pilot/comparison.md

The evaluation test command should report 62 passing tests. Canonical traces are intentionally immutable. For another run in the same checkout, provide a fresh output directory:

python -m evaluation.compare_pilot --outdir .local-output/eval/m2-pilot-rerun

The submitted evidence is preserved separately under `evaluation/results/m2-pilot/`; local reruns never overwrite those committed artifacts.

Team Contributions

1. Neha Kashyap — GitHub: @nkashp
   As owner of the deterministic core and safety workstream for the Claim Denial Triage Agent, Neha built the four foundational files that the rest of the system depends on: router.py (the deterministic route decision), schema.py (the shared contracts), package.py (the final review package, which stays PENDING_REVIEW until a human acts), and validator.py (the safety validation layer with explicit failure and escalation reasons). Neha wrote a test file for each of them. Then took on the second-labeller role for Dataset B, independently labeling the 12 cases to create set_b_development.labeler_b.json.
   Associated Pull Requests: 8, 14, 15, 16, 17, 18, 25, 27, 31, 32, 36, 40, 42

2. Hriday Pradhan — GitHub: @hridaypradhan
   As owner of the agent runtime, integration, and auditability workstream, Hriday built the bounded execution layer that turns routed claims into safe, reviewable outcomes. This work included the provider-independent model interface, deterministic scripted model, strict action parser, bounded planning and repair loops, model/tool/step/time budgets, controlled retries and timeouts, explicit completion and manual-review states, append-only runtime traces, and command-line runners. He then integrated the runtime with the team's router, retrieval dispatcher, shared contracts, safety validator, and output-package boundary, ensuring that decisions use evidence retrieved during the same run and remain `PENDING_REVIEW` for human approval. Hriday also implemented the canonical trace adapter and reproducible run configuration used to audit model requests, tool calls, validation results, terminal reasons, budgets, and configuration hashes. For the Milestone 2 evaluation, he built the S0 evaluation adapter that runs all 35 cases across both datasets, added deterministic evidence-grounded plans for agent-routed cases, coordinated and recorded the adjudication of Dataset A and Dataset B, preserved both labelers' original files, and added end-to-end evaluation tests and documentation. He also reviewed and integrated teammate-owned boundaries, tracked cross-team blockers, and verified the pilot failure analysis, which showed that two missed escalations occurred during routing before the bounded runtime executed.
   Associated Pull Requests: [#1], [#10], [#33], [#35], [#43]
3. Silas Sangmin - @Silas281
   Silas led the evidence and retrieval workstream. He built the versioned knowledge corpus and read-only tools for retrieving policy requirements, encounter notes, denial codes, and payer profiles. His implementation matches policies by payer, procedure, and service date, and provides citable passages with source hashes. He also expanded the corpus with conflicting evidence and instruction-like content while preserving existing passage IDs. For the Milestone 2 pilot, he served as the second labeler for all 23 Dataset A cases. He used an assistant to draft labels, reviewed them himself without seeing the first labeler's file, and flagged five cases for adjudication. He added the escalate-all baseline and compared the scripted agent with both baselines across both datasets. The evaluation covered labeler agreement, Cohen's kappa, tool use, citation accuracy, and a documented failure case. Silas also proposed the shared tool-output contract and reviewed teammates' work on tool failures, retry tracing, passage ordering, validation, and annotation rules. He maintained blind labeling for Dataset B, checked labels against the corpus, verified evaluation results, and posted the final pilot findings. He also moderated stakeholder meetings with Mokshitha.
   Associated Pull Requests: [4], [18],[20],[22],[26],[27],[36],[39],[41],[44]
4. Sachin Kothandaraman — GitHub: @Sachin-Kothandaraman-del
   As the owner of the evaluation and observability workstream, Sachin defined how the team would measure the Claim Denial Triage Agent and ensured that every reported number could be traced and reproduced. He wrote the trace and run-configuration requirements for evaluation. He also wrote the annotation rubric, which was frozen before any system output was scored. The rubric defines the routing rules, what counts as missing versus ruled-out evidence, and the critical safety errors. He built Dataset A with 23 rule and safety cases across all ten root causes. He also built the baseline pilot scoring harness, which reports joint accuracy, resolution coverage, selective accuracy, deadline accuracy and safety errors, each with Wilson confidence intervals. He then built Dataset B, 12 evidence-dependent medical-necessity cases (CO-50) covering dated policy versions, conflicting notes, an injected instruction and missing policies. He served as the first labeler for both datasets. He added code for labeler agreement, Cohen's kappa and system comparison, which was cross-checked against the final comparison in #44. Sachin reviewed teammates' work on the scripted-agent runtime, the knowledge corpus, adjudication and pilot results. He ran S0, S1 and S2 on both datasets and posted the final tables, safety counts and failure case to #12. He also wrote the report's evaluation results and failure analysis sections, which identify the missed-escalation failure in routing and propose a fix.
   Associated Pull Requests: [#8], [#31], [#32], [#40], [#45]